

Contents

CLAUDIU user manual	1
Starting	1
Tabs	1
The launcher	1
The conversation	1
Prompting	2
Approvals	2
Status strip and warnings	2
Records	2
Security notes	2

CLAUDIU user manual

CLAUDIU (working codename) is a local, browser-based client for AI coding agents that speak the Agent Client Protocol (ACP). It runs a small Python server on your machine, spawns the agent’s ACP adapter as a plain subprocess, and renders the structured conversation — messages, thinking, tool calls with diffs, permission requests — in your browser. Nothing is screen-scraped; everything you see comes from typed protocol data, and every byte that crosses the wire is recorded.

Starting

```
python -m claudiu
```

Options: `--port N` (default 0 = OS-assigned), `--profiles DIR` (default `agents/`), `--records DIR` (default `~/.claudiu/records`), `--no-drift-online` (skip the update check). The server prints a URL with a one-time token — open it; the token becomes a cookie and the address bar cleans itself. Each launch gets a fresh token; old URLs die with the server.

Tabs

Each session is a tab (Alt+1...9 switch, Alt+N opens the launcher, × closes and ends the session). The tab shows the agent’s own title for the session once it sets one, and a dot: green ready, pulsing blue working, red failed or disconnected. The status strip, selectors and composer always belong to the active tab; approval dialogs from any tab pop up wherever you are, labelled with the session they belong to.

The launcher

Pick an **agent** (from `agents/*.toml` profiles; an agent whose adapter is not installed shows “adapter missing” with the install hint) and a **project directory** — the session’s working directory and, importantly, its *file-access boundary*: the agent can only read/write inside it through this client. Known caveats of the selected agent are listed right there, followed by the **runtime** the adapter will be pointed at: for Claude, `CLAUDE_CODE_EXECUTABLE` resolves to your installed `claude` when there is one on `PATH`. The adapter bundles its own, older Claude CLI, which the API may refuse for newer models (“version 2.1.251 or newer is required” was the 2026-09-01 symptom); with no `claude` installed the bundled copy is used. The older adapter may then print harmless “Unexpected case” lines on stderr (shown as collapsible rows) for message types newer than it knows. **Find resumable sessions** asks the agent which of its own sessions exist for that directory (a throwaway adapter is started and closed for the query) and offers a **Resume** button per session.

The conversation

- **Agent text** streams as it is produced; *thinking* appears dimmed and italic; your prompts are boxed.
- **Tool calls** are one row per call, updated in place as the agent reports progress (pending → in progress → completed/failed, colour-coded edge). Edits arrive as real diffs (+/− lines); text output and file locations open under “details”.

- **Plan** entries (the agent’s todo list) fill the panel above the composer.
- **Turn ends** are marked with the protocol’s stop reason.
- The {} button on special rows reveals the raw protocol frame behind them — the escape hatch into the session record.

Prompting

Enter sends; Shift+Enter inserts a newline. The Send button is enabled only while the session is ready (not during the agent’s turn, not before startup completes). **Stop** cancels the current turn. Typing / first opens the command palette listing the agent’s own slash commands; picking one only fills the composer — nothing is sent until you press Send, and the agent’s response to a command is always rendered.

Approvals

When the agent wants to do something that needs permission, a dialog shows the tool call (with diff when provided) and the agent’s own options — allow once, allow always, reject — as buttons (digits 1–9 work too). There is no Escape-to-dismiss: an explicit choice is required. Unanswered requests are auto-rejected after a timeout (default one hour) and marked as fail-safe rejections.

Status strip and warnings

The strip shows the session state, the current permission mode, a **context gauge** (tokens used / window size, percentage, and cost when the agent reports it) and the current model. Selectors next to the composer switch **mode**, **model** (with the agent’s description — Claude’s “Default” is Opus) and any **configuration options** the agent advertises. While a turn runs, a pulsing “agent working... 42s · last activity 3s ago (tool_call: Edit hello.py)” row sits at the end of the conversation — the elapsed time tells you the turn is alive, the last-activity part tells you whether the agent is still producing events (it turns amber after a minute of silence). Two chips can appear and should not be ignored:

- **drift** — the agent sent protocol data newer than this client’s pinned ACP schema (hover for details). The client keeps working and keeps recording; the flag means an update to the client is due.
- **anomalies** — something out of order happened (malformed frame, adapter crash, rejected command...); the conversation shows the details inline with raw-frame access.

Records

Every session appends to `<records-dir>/<session>.jsonl`: each protocol frame verbatim (before any interpretation) plus every client action — prompts, permission answers, file-access decisions with the policy that made them. This is the audit trail and the ground truth; the UI’s **raw_ref** numbers index into it.

Security notes

The server binds 127.0.0.1 only; every request needs the launch token; foreign Host/Origin headers are refused. Agent file access is confined to the project directory (symlinks resolved). Adapter subprocesses run with a scrubbed environment (plus the profile’s declared runtime resolution and settings, written as the first record of the session) and die with their session. The record files never contain your token — but they do contain your conversation, so treat the records directory accordingly.